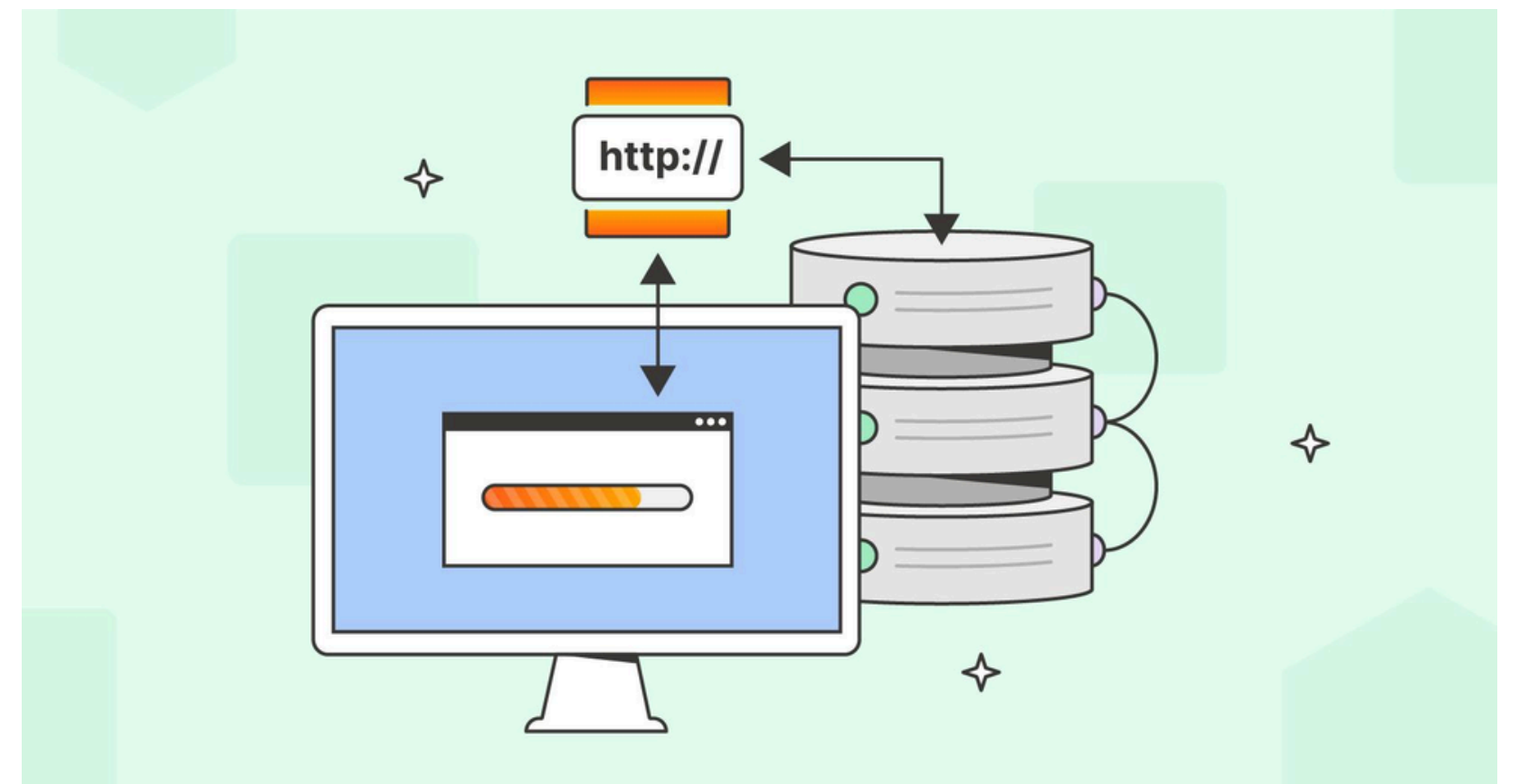


HTTP/HTTPS



Протокол HTTP

HTTP расшифровывается как Hypertext Transfer Protocol, протокол передачи гипертекста. Сейчас это один из самых популярных протоколов интернет, основа Web.

HTTP находится на прикладном уровне в моделях OSI и TCP/IP.

Концепцию Web предложил в 1989 году Тим Бернерс-Ли из Европейского центра ядерных исследований (ЦЕРН). Основные компоненты Web по предложению Тима Бернерс-Ли:

- Язык гипертекстовой разметки страниц HTML.
- Протокол передачи гипертекстовых страниц HTTP.
- Web-сервер.
- Текстовый web-браузер.

Сейчас Web устроен более сложно, браузеры поддерживают не только текст, но и изображения, видео, могут запускать код на JavaScript и многое другое.

Протокол HTTP используется браузером для того, чтобы загрузить с Web-сервера HTML страницы и другие ресурсы, которые нужны для показа страниц. Также HTTP сейчас активно применяется в API.

Uniform Resource Locator

Важную роль в работе HTTP играет Uniform Resource Locator, сокращенно URL – единообразный определитель местонахождения ресурса. Именно URL используется для того, чтобы указать, к какой странице мы хотим получить доступ.

- **`http://www.myserver.ru/mycourse/web320`**

Протокол Адрес сервера Страница

URL состоит из трех основных частей:

- **Название** протокола, в примере на протокол HTTP.
- **Адрес сервера**, на котором размещен ресурс. Можно использовать IP-адрес или доменное имя. Адрес сервера отделяется от названия протокола двоеточием и двумя слешами.
- Адрес ресурса на сервере. Это может быть HTML-страница, изображение, видео или ресурс другого типа. В примере на рисунке адрес страницы: `/mycourse/web320`.

В URL не обязательно использовать только протокол HTTP, вот примеры с другими протоколами:

- `https://ya.ru`
- `ftp://example.com`

URL может включать достаточно большое количество других компонентов, кроме протокола, адреса сервера и адреса ресурса. Более подробно почитать о них можно в документе:

- [RFC 1738, Uniform Resource Locators \(URL\)](#).

Версии протокола HTTP

В ЦЕРН в 1991 году разработали экспериментальную версию протокола HTTP 0.9. Первая открытая версия HTTP 1.0 появилась в 1996 году, она описана в документе [RFC 1945](#).

Почти сразу после HTTP 1.0, в 1997 году, вышла обновленная версия протокола HTTP 1.1. В этой версии добавили кэширование, постоянное соединение, аутентификацию и некоторые другие возможности. Документы RFC, описывающие HTTP 1.1, несколько раз обновлялись, сейчас действует [RFC 9112](#) от 2022 года. Версия HTTP 1.1 используется сейчас.

В 2015 году вышла версия HTTP/2 (описана в [RFC 9113](#)), основная задача которой – повышение производительности. Самая последняя версия HTTP/3, описанная в [RFC 9114](#), использует на транспортном уровне [протокол QUIC](#) вместо TCP.

Версии HTTP/1.1, HTTP/2 и HTTP/3 отличаются форматом передачи данных по сети, но логика работы у них одинаковая. Эта логика описана в документе [RFC 9110 HTTP Semantics](#).

HTTP передает данные по сети в открытом виде, что не очень хорошо с точки зрения безопасности. Шифрование данных используется в протоколе HyperText Transfer Protocol Secure, сокращенно HTTPS (будет рассмотрен далее).

Протокол HTTP

Протокол HTTP работает в режиме запрос-ответ. Клиент, например, браузер, передает на сервер запрос к определенному ресурсу, например, Web-странице. Сервер в ответ отправляет клиенту этот ресурс или сообщение об ошибке, если ресурс передать нельзя.



HTTP/1.1 использует текстовый режим работы, то есть от клиента к серверу и обратно передаются обычные строки. Это просто и удобно для людей, которые могут понять, что именно передается без использования дополнительных инструментов. Однако для компьютеров это не самый удобный формат, т.к. приходится заниматься парсингом строк. Поэтому HTTP/2 и HTTP/3 используют бинарный режим работы.

На транспортном уровне HTTP использует протокол TCP (кроме HTTP/3, в котором применяется QUIC), порт Web-сервера по-умолчанию: 80.

Запрос HTTP

Запрос HTTP состоит из трех основных частей:

- Запрос
- Заголовки (не обязательно)
- Тело сообщения (не обязательно)

Пример простого запроса HTTP в текстовом режиме:

GET /mycourses/python/classes/1.1

Host: myhost.ru

В первой строке указывается сам запрос, который также состоит из трех частей:

- Метод HTTP GET указывает, какое действие требуется выполнить с ресурсом. В примере метод GET говорит о том, что мы хотим получить (загрузить) ресурс.
- Адрес ресурса /mycourses/python/classes – путь к странице на Web-сервере, которую мы хотим загрузить.
- Версия протокола HTTP/1.1.

Во второй строке указывается заголовок Host. Этот заголовок является обязательным в версии HTTP/1.1, в нем задается доменное имя сервера, к которому направлен запрос. Сейчас активно используется виртуальный хостинг: один Web-сервер на одном IP-адресе может обслуживать несколько сайтов с разными доменными именами. Чтобы понять, какому именно сайту предназначен запрос, нужно указать доменное имя сайта в заголовке Host.

В HTTP используется следующий формат заголовка:

Название заголовка: значение заголовка

В примере название заголовка Host, значение – myhost.ru (доменное имя сайта, к которому направлен запрос).

Если заголовков в запросе несколько, то каждый указывается в отдельной строке.

Версии протокола HTTP

В ЦЕРН в 1991 году разработали экспериментальную версию протокола HTTP 0.9. Первая открытая версия HTTP 1.0 появилась в 1996 году, она описана в документе [RFC 1945](#).

Почти сразу после HTTP 1.0, в 1997 году, вышла обновленная версия протокола HTTP 1.1. В этой версии добавили кэширование, постоянное соединение, аутентификацию и некоторые другие возможности. Документы RFC, описывающие HTTP 1.1, несколько раз обновлялись, сейчас действует [RFC 9112](#) от 2022 года. Версия HTTP 1.1 используется сейчас.

В 2015 году вышла версия HTTP/2 (описана в [RFC 9113](#)), основная задача которой – повышение производительности. Самая последняя версия HTTP/3, описанная в [RFC 9114](#), использует на транспортном уровне [протокол QUIC](#) вместо TCP.

Версии HTTP/1.1, HTTP/2 и HTTP/3 отличаются форматом передачи данных по сети, но логика работы у них одинаковая. Эта логика описана в документе [RFC 9110 HTTP Semantics](#).

HTTP передает данные по сети в открытом виде, что не очень хорошо с точки зрения безопасности. Шифрование данных используется в протоколе HyperText Transfer Protocol Secure, сокращенно HTTPS (будет рассмотрен далее).

Методы HTTP

Метод HTTP говорит о том, какое действие с ресурсом мы хотим совершить. В примере запроса мы видели метод GET, который предназначен для получения ресурса. Кроме GET в HTTP есть и другие методы, наиболее важные из которых определены в документе [RFC 9110 HTTP Semantics](#).

Название метода	Описание метода
GET	Запрос на передачу ресурса.
HEAD	Запрос на передачу ресурса, но сам ресурс в ответе не передается, только заголовки.
POST	Передача данных на сервер для обработки указанного ресурса.
PUT	Размещение ресурса на сервере (если такой ресурс уже есть на сервере, то он замещается).
PATCH	метод запроса в HTTP для внесения частичных изменений в существующий ресурс.
DELETE	Удаление ресурса на сервере.
CONNECT	Установка соединения с сервером на основе ресурса.
OPTIONS	Запрос поддерживаемых методов HTTP для ресурса и других параметров коммуникации.
TRACE	Запрос на трассировку сообщения: сервер должен включить в свой ответ исходный запрос, на который он отвечает. Это полезно, когда запрос проходит через промежуточные устройства, которые могут изменить запрос, например, добавить заголовки.

Полный список всех существующих методов HTTP находится в документе [Hypertext Transfer Protocol \(HTTP\) Method Registry](#), который сопровождается организацией [Internet Assigned Numbers Authority \(IANA\)](#).

Следует отметить, что методы HTTP и соответствующие им действия – это соглашения, которые желательно, но не обязательно соблюдать. Вполне возможно, что некоторая реализация Web-сервера в ответ на запрос GET будет удалять запрошенный ресурс. Однако большая часть реализаций соблюдает соглашения.

На практике в Web чаще всего используются два метода: GET для запроса Web-страниц и POST для передачи данных из браузера на сервер, например, после заполнения формы.

Ответ HTTP

Ответ HTTP, как и запрос, состоит из трех частей:

- Статус ответа
- Заголовки (не обязательно)
- Тело сообщения (не обязательно)

Пример ответа на запрос из ранее:

HTTP/1.1 200 OK

Content-Type: text/html; charset=UTF-8

Content-Length: 3128

Текст Web-страницы...

В HTTP/1.1 ответ, также как и запрос, представляет собой обычные текстовые строки.

Первая строка ответа состоит из двух частей:

- Версия протокола, в примере **HTTP/1.1**. Версию указывать не обязательно.
- Код статуса ответа, в примере **200 OK**, запрос обработан успешно.

Ответ содержит два заголовка, которые следуют после строки с кодом статуса:

- **Content-Type**, тип содержимого, которое передается в теле сообщения. Значение "text/html; charset=UTF-8" означает, что в теле сообщения страница HTML в кодировке UTF-8.
- **Content-Length**, размер содержимого, в байтах. В примере размер страницы в теле сообщения 3128 байт.

После заголовков в теле ответа находится запрошенная Web-страница (в примере она не показана для сокращения объема). Тело сообщения отделено от заголовков пустой строкой.

Коды статусов ответов HTTP

Первая строка ответа HTTP содержит код статуса ответа – число в диапазоне от 100 до 599, которое характеризует результат выполнения запроса. Возможные коды статусов ответов описаны в документе [RFC 9110 HTTP Semantics](#).

Коды статусов ответов разделены на пять классов, которые определяются по первой цифре кода:

- 1XX (информация): запрос получен, обработка продолжается.
- 2XX (успешное выполнение): запрос был успешно принят и понят.
- 3XX (перенаправление): для выполнения запроса необходимо предпринять дополнительные действия.
- 4XX (ошибка клиента): запрос содержит синтаксическую ошибку или не может быть выполнен.
- 5XX (ошибка сервера): запрос от клиента оформлен правильно, но при его обработке произошла ошибка на стороне сервера.

Полный список кодов ответов с описанием можно посмотреть в разделе [«Status Codes» документа RFC 9110](#).

Протокол HTTPS

HTTPS (от англ. HyperText Transfer Protocol Secure) – это безопасный протокол передачи данных, который поддерживает шифрование посредством криптографических протоколов SSL и TLS, и является расширенной версией протокола HTTP. Чтобы лучше понять, что же значит HTTPS, разберемся со всем по порядку.

Как мы уже знаем HTTP использовался только как протокол передачи гипертекста (текста с перекрёстными ссылками). Однако позже стало понятно, что он отлично подходит для передачи данных между пользователями. Протокол был доработан для новых задач и стал использоваться повсеместно.

Несмотря на свою функциональность у **HTTP есть один очень важный недостаток** — незащищённость. Данные между пользователями передаются в открытом виде, злоумышленник может вмешаться в передачу данных, перехватить их или изменить. Чтобы защитить данные пользователей, был создан протокол HTTPS.

HTTPS работает благодаря SSL/TLS-сертификату. SSL/TLS-сертификат — это цифровая подпись сайта. С её помощью подтверждается его подлинность. Перед тем как установить защищённое соединение, браузер запрашивает этот документ и обращается к центру сертификации, чтобы подтвердить легальность документа. Если он действителен, то браузер считает этот сайт безопасным и начинает обмен данными. Вот откуда взялась и что означает S в HTTPS.

Чем отличается SSL от TLS

Небольшая историческая справка:)

Для защиты интернет-соединения в 90-х годах компания Netscape создала SSL-сертификат. У первых версий этого сертификата было много недостатков. За несколько лет вышло три версии SSL. Огромный скачок в его усовершенствовании произошел в 1999 году при совместной работе с компанией Consensus Development. Кроме серьёзных доработок, сертификат получил новое название — TLS (что значит — защита транспортного уровня). Несмотря на новое название, пользователи всё равно применяли привычное понятие SSL. Разработчики встали на сторону пользователей и не стали акцентировать внимание на наименовании, поэтому часто можно увидеть двойное название этого сертификата (SSL/TLS), или просто SSL. Однако в официальной документации используется аббревиатура TLS.

Система HTTPS похожа на провод, который состоит из двух слоёв: медная сердцевина и оболочка. Медная сердцевина — основная часть провода, по которой идёт ток. Оболочка защищает контакты от внешних воздействий. Так, медная сердцевина — это HTTP-протокол, а защитная оболочка — это SSL-сертификат. Такое сотрудничество создаёт безопасное HTTPS-соединение.

Ключи шифрования

Кроме подтверждения подлинности сайта, SSL-сертификат шифрует данные. После того как браузер убедился в подлинности сайта, начинается обмен шифрами. Шифрование HTTPS происходит при помощи симметричного и асимметричного ключа. Вот что это значит:

- **Асимметричный ключ** — каждая сторона имеет два ключа: публичный и частный. Публичный ключ доступен любому. Частный известен только владельцу. Если браузер хочет отправить сообщение, то он находит публичный ключ сервера, шифрует сообщение и отправляет на сервер. Далее сервер расшифровывает полученное сообщение с помощью своего частного ключа. Чтобы ответить пользователю, сервер делает те же самые действия: поиск публичного ключа собеседника, шифрование, отправка.
- **Симметричный ключ** — у обеих сторон есть один ключ, с помощью которого они передают данные. Между двумя сторонами уже должен быть установлен первичный контакт, чтобы браузер и сервер знали, на каком языке общаться.

Чтобы установить HTTPS-соединение, браузеру и серверу надо договориться о симметричном ключе. Для этого сначала браузер и сервер обмениваются асимметрично зашифрованными сообщениями, где указывают секретный ключ и далее общаются при помощи симметричного шифрования.

Ключи шифрования

Итак, какова функция протокола HTTPS?

1. Шифрование. Информация передаётся в зашифрованном виде. Благодаря этому злоумышленники не могут украсть информацию, которой обмениваются посетители сайта, а также отследить их действия на других страницах.
2. Аутентификация. Посетители уверены, что переходят на официальный сайт компании, а не на дубликат, сделанный злоумышленником.
3. Сохранение данных. Протокол фиксирует все изменения данных. Если злоумышленник всё-таки пытался взломать защиту, об этом можно узнать из сохранённых данных.

Также стоит упомянуть, какой порт используется протоколом HTTPS по умолчанию. HTTPS использует для подключения 443 порт — его не нужно дополнительно настраивать

Схема работы HTTPS (в части secure)

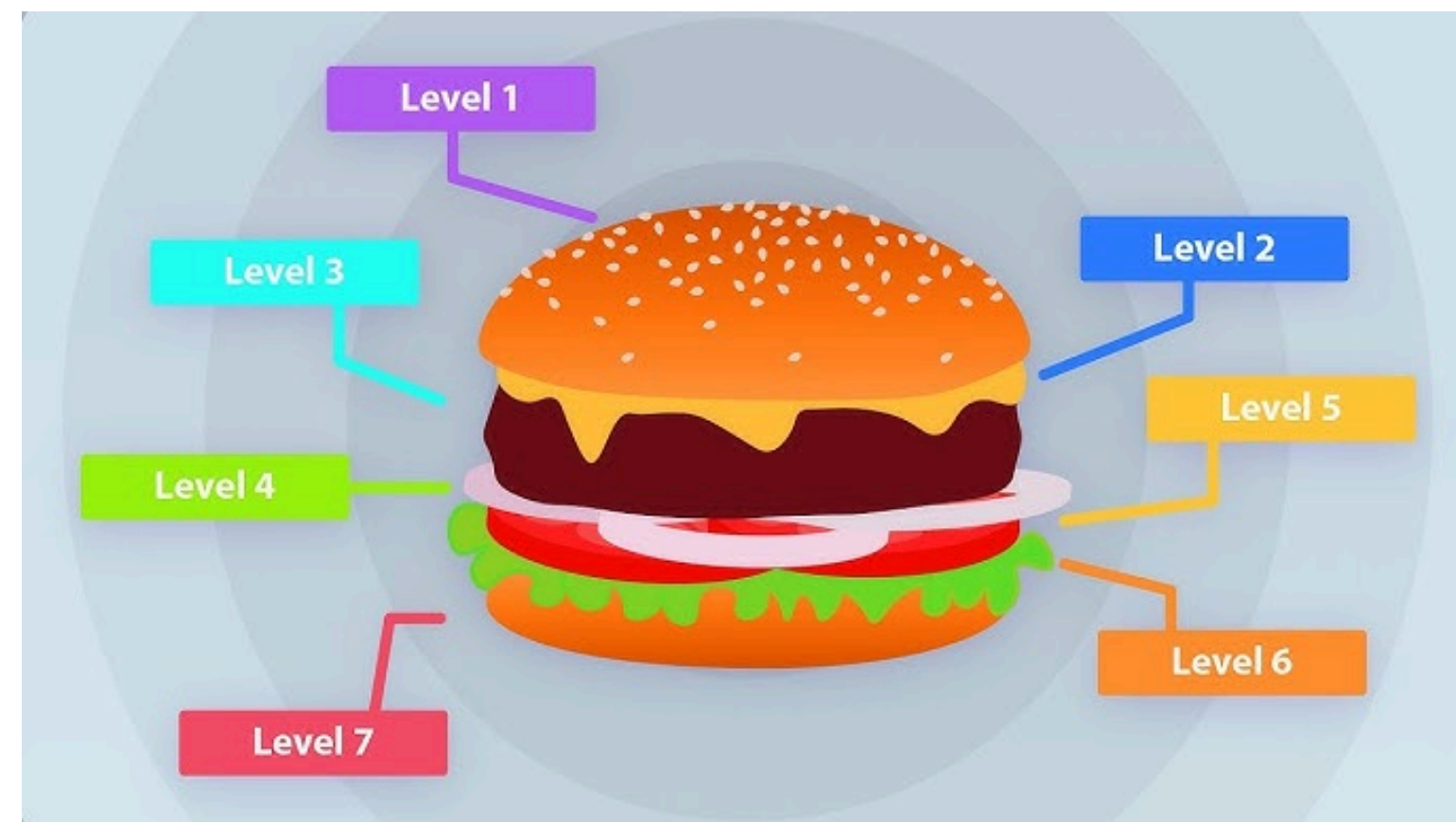
Итак, протокол HTTPS предназначен для безопасного соединения. Чтобы понять, как устанавливается это соединение и как работает HTTPS, рассмотрим механизм пошагово.

Для примера возьмём ситуацию: пользователь хочет перейти на сайт yandex.ru, который работает по безопасному протоколу HTTPS.

1. Браузер пользователя просит предоставить SSL-сертификат.
2. Сайт на HTTPS отправляет сертификат.
3. Браузер проверяет подлинность сертификата в центре сертификации.
4. Браузер и сайт договариваются о симметричном ключе при помощи асимметричного шифрования.
5. Браузер и сайт передают зашифрованную информацию.



Модель OSI



Краткая история разработки модели OSI

История разработки OSI началась с небольшой группы ученых, во главе которой стояли Майк Канепа и Чарльз Бакман. В начале и середине 1970-х годов основное внимание группы было сосредоточено на проектировании и разработке прототипов систем для компании Honeywell Information System. А в середине 1970-х группа поняла, что для поддержки машин с базами данных распределенного доступа и их взаимодействия необходима более структурированная коммуникационная архитектура.

Honeywell

Результатом исследований и работы над проектированием собственного решения стала разработка в 1977 году многоуровневой архитектуры, известной как архитектура распределенных систем HDSA (Honeywell Distributed System Architecture). Этот проект создавался, чтобы предоставить виды протоколов «процессор-процессор» и «процессор-терминал», необходимые для взаимодействия произвольного количества машин и произвольного количества людей. Это должно было стать основой для создания системных приложений (Чарльз Бакман, [интервью](#) Джеймса Л.Пелки).

Вашингтон, округ Колумбия, март 1978 года

С 28 февраля по 2 марта 1978 года в Вашингтоне проходило собрание ISO (Международной организации по стандартизации), где команда Honeywell презентовала свое решение ISO. На встрече собралось множество делегатов из десяти стран и наблюдатели из 4 международных организаций. На этом совещании было достигнуто соглашение, что многоуровневая архитектура HDSA удовлетворяет большинству требований и что ее можно будет расширить позже.



Краткая история разработки модели OSI

Для дальнейшей работы над усовершенствованием модели было решено собрать рабочие группы. Их главной целью было составление общего международного архитектурного положения.

Модель, которую представили на собрании, состояла из шести слоев, куда изначально не входил нижний, физический уровень. И здесь вступает в игру Юбер Циммерманн, председатель OSI и глава архитектурной группы, который и предложил включить в модель физический уровень. Необходимо было узнать, как подавать импульсы на провода. Чарльз Бакман отмечает, что Юбер был одним из самых важных людей в этом комитете, с точки зрения его вклада в работу.

Начиная с 1977 года, ISO провела программу по разработке общих стандартов и методов создания сетей, но аналогичный процесс появился в некоммерческой организации по стандартизации информационных и коммуникационных систем (ЕСМА) и Международном консультативном комитете по телеграфу и телефону (CCITT). Делегаты от этих групп присутствовали на собраниях ISO, и все они работали над одной целью. Позже CCITT приняла документы, которые почти идентичны документам ISO, и группа стала сотрудничать с ISO.

В 1983 году документы CCITT и ISO были объединены, чтобы сформировать Базовую эталонную модель взаимодействия открытых систем или просто модель OSI. Общий документ был опубликован в 1984 году как стандарт ISO 7498.

Сетевая модель OSI

Как вы могли понять из истории, это набор правил, который описывает процесс взаимодействия устройств по сети. OSI выступает первой стандартной моделью в области сетевых коммуникаций.

Референтная модель OSI

7. Прикладной

6. Уровень представления

5. Сеансовый

4. Транспортный

3. Сетевой

2. Канальный

1. Физический

По модели процесс передачи данных по сети происходит постепенно от одного уровня к другому. На каждом из них используются информация с прошлого уровня и определенные протоколы. Главными героями здесь выступают устройства отправителя и получателя, а также сами передаваемые данные. И как раз процесс обмена информации между устройствами определяет модель OSI.

На физическом уровне информация предстает в виде битов, а на прикладном она отражается в более привычном для нас виде, в виде данных. Существует два процесса перехода от первого уровня к седьмому и наоборот. Первый – это инкапсуляция, когда данные отправляются с устройства и переводятся в биты. Второй – декапсуляция, обратный переход, когда биты трансформируются в данные.

Разбираемся, что конкретно делают уровни, и что же там происходит. Смотрим на модель снизу вверх.



Сетевая модель OSI. Уровни

Референтная модель OSI

7. Прикладной

6. Уровень представления

5. Сеансовый

4. Транспортный

3. Сетевой

2. Канальный

1. Физический

Уровень 1: Физический

Начнем с уровня 1. Здесь происходит обмен оптическими, электрическими или радиосигналами между устройствами отправителя и получателя.

На этом уровне железо не распознает данные в классическом для нас виде (картинки, текст, видео), но оно понимает биты (единицы и нули) и работает только с сигналами. Таким оборудованием выступают концентраторы, медиаконвертеры или репитеры. Здесь информация или биты передаются либо по проводам, кабелям, либо без них, например через Bluetooth, Wi-Fi.

Когда возникает проблема с сетью, многие специалисты сразу же обращаются к физическому уровню, чтобы проверить, например, не отключен ли сетевой кабель от устройства.



Сетевая модель OSI. Уровни

Референтная модель OSI

7. Прикладной

6. Уровень представления

5. Сеансовый

4. Транспортный

3. Сетевой

2. Канальный

1. Физический

Уровень 2: Канальный

Мы прошли первый уровень. Что же дальше? Если в локальной сети находится более двух устройств, то необходимо определить, куда конкретно направлять информацию. Этим занимается как раз канальный уровень, принимающий на себя важную роль адресации.

Второй уровень принимает биты и трансформирует их в кадры (фреймы). Здесь существуют MAC-адреса (Media Access Control), которые необходимы для идентификации устройств. На втором уровне происходит еще проверка на ошибки, и исправление информации, а также управление ее передачей. Этим занимается LLC (Logical Link Control).

На канальном уровне работают уже более умные железки – коммутаторы. Их задачей является передача кадров от одного устройства другому, используя MAC-адреса.



Сетевая модель OSI. Уровни

Референтная модель OSI

7. Прикладной

6. Уровень представления

5. Сеансовый

4. Транспортный

3. Сетевой

2. Канальный

1. Физический

Уровень 3: Сетевой

На третьем уровне происходит маршрутизация трафика. Этим занимаются такие устройства, как роутеры или маршрутизаторы.

На сетевом уровне работает протокол ARP (Address Resolution Protocol), который определяет соответствие между логическим адресом сетевого уровня (IP) и физическим адресом устройства (MAC). Здесь пересылаемая информация выступает уже в виде пакетов, состоящих из заголовка и поля данных.

Информация об известных IP и MAC-адресах хранится в виде таблицы (ARP-таблица) с данными, что позволяет устройствам не тратить время на повторную идентификацию.



Сетевая модель OSI. Уровни

Референтная модель OSI

7. Прикладной

6. Уровень представления

5. Сеансовый

4. Транспортный

3. Сетевой

2. Канальный

1. Физический

Уровень 4: Транспортный

Четвертый уровень получает пакеты и передает их по сети. Он отвечает за установку соединения, надежность и управление потоком. Блоки данных делятся на отдельные фрагменты, размеры которых зависят от используемого протокола. Главными героями тут выступают 2 протокола TCP (Transmission Control Protocol) и UDP (User Datagram Protocol). В чем их отличие и когда их применять?

UDP-протокол используется с данными, для которых потери не так критичны, например, мультимедиа-трафик. Для них более заметна будет задержка, поэтому UDP обеспечивает связь без установки соединения. Во время передачи данных с помощью протокола UDP, пакеты делятся уже на автономные датаграммы. Они могут доставляться по разным маршрутам и в разной последовательности.



Сетевая модель OSI. Уровни

Референтная модель OSI

7. Прикладной

6. Уровень представления

5. Сеансовый

4. Транспортный

3. Сетевой

2. Канальный

1. Физический

Уровень 5: Сеансовый

Уровни с пятого по седьмой уже работают с чистыми данными. И здесь за дело берутся не сетевые инженеры, а разработчики.

Сеансовый уровень, исходя из названия, отвечает за поддержание сеанса или сессии. Он координирует коммуникацию между приложениями и отвечает за установление, поддержание и завершение связи, синхронизацию задач и сам обмен информацией. Примером для пятого уровня можно назвать созвон в Zoom или прямой эфир на YouTube. Во время сессии необходимо обеспечивать синхронизированную передачу аудио и видео для всех участников, а также поддерживать саму связь. За это как раз отвечают протоколы сеансового уровня (RPC, H.245, RTCP).



Сетевая модель OSI. Уровни

Референтная модель OSI

7. Прикладной

6. Уровень представления

5. Сеансовый

4. Транспортный

3. Сетевой

2. Канальный

1. Физический

Уровень 6: Уровень представления

Шестой уровень подготавливает информацию для последнего и преобразует (сжимает, кодирует, шифрует) их в понятный язык для пользователя или машины. Например, если вы отправляете картинку, то она сначала приходит в виде битов, а потом трансформируются в JPEG, GIF или другой формат.

Уровень 7: Прикладной

Верхний уровень модели OSI – это прикладной. С помощью своих протоколов он отображает данные в понятном конечном пользователю формате. Сюда входят такие технологии, как HTTP, DNS, FTP, SSH и многое другое. Почти каждый человек ежедневно взаимодействует с протоколами прикладного уровня.



Сетевая модель OSI. Как это все работает?

Референтная модель OSI

7. Прикладной

6. Уровень представления

5. Сеансовый

4. Транспортный

3. Сетевой

2. Канальный

1. Физический

Чтобы информация могла быть передана по сети от устройства к устройству, данные должны пройти семь кругов, а точнее уровней по модели OSI.

Информация передается с уровня 7 вниз на уровень 1 от отправителя, а затем передается с уровня 1 на уровень 7 на устройстве получателя.

Примером передачи данных по модели OSI является приложение электронной почты. Когда пользователь отправляет письмо, оно приходит на уровень представления с использованием определенного протокола (SMTP для исходящей электронной почты). Уровень представления сжимает информацию и отправляет сообщение на сеансовый, который открывает сессию для связи между устройством отправителя и исходящим сервером.

Далее вступает в силу транспортный уровень, где сегментируются полученные данные. Затем сетевой уровень разбивает сегменты на пакеты и отправляет их на канальный уровень, где они разбиваются на фреймы. Фреймы переходят на физический уровень, где информация преобразуется в биты и передается через физическую среду, беспроводные соединения или кабели.

Когда сообщение доходит до получателя, происходит обратный процесс, где информация переходит из битовых единиц и нулей в сообщение на почте получателя. Как-то так.



Сетевая модель OSI. Что же дальше?

Если кратко разбирать, что произошло дальше, то в 1970-90-х существовало два конкурирующих стандарта: протокол TCP/IP и OSI. Несмотря на годы разработки, серьезные усилия со стороны лидеров отрасли, правительств и ученых, OSI была отвергнута на практике, и TCP/IP стал стандартом де-факто для всего интернет-трафика.

OSI была попыткой отрасли убедить участников согласовать общие сетевые стандарты. В течение периода в конце 1980-х и начале 1990-х инженеры, организации и страны поляризовались по вопросу о том, какой стандарт, модель OSI или TCP/IP сделает компьютерные сети надежными. Однако в то время как OSI разработала свои сетевые стандарты в конце 1980-х, TCP/IP стал широко использоваться для межсетевого взаимодействия. Строгую многоуровневую структуру OSI интернет-защитники считали неэффективной.

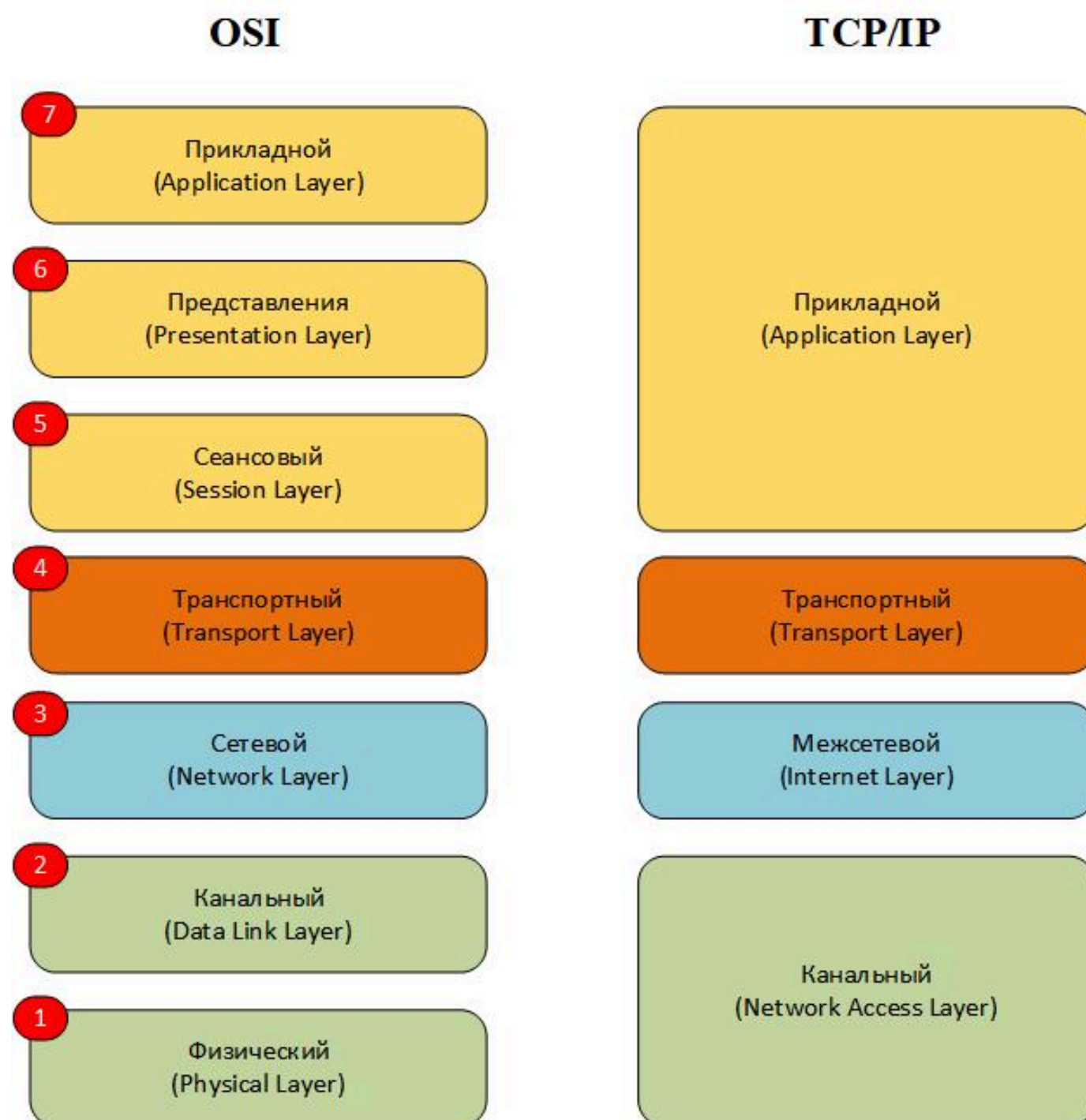
Модель OSI все еще используется в качестве эталона, однако протоколы, изначально задуманные для этой модели, не приобрели популярности. Некоторые инженеры утверждают, что эталонная модель по-прежнему актуальна для облачных вычислений. Другие говорят, что исходная модель OSI не соответствует сегодняшним сетевым протоколам, и предлагают вместо этого упрощенный подход.

В презентации была описана история появления модели OSI и принцип ее работы. Она подходит для теоретического понимания сетевого стека, поскольку это базовая и обязательная технология для работы с сетями, но ее, на самом деле, не так просто использовать на практике. В настоящее время применяют модель TCP/IP, на которой работает Интернет. Она имеет аналогичную многоуровневую структуру, но не такую сложную, потому что объединяет некоторые уровни OSI.



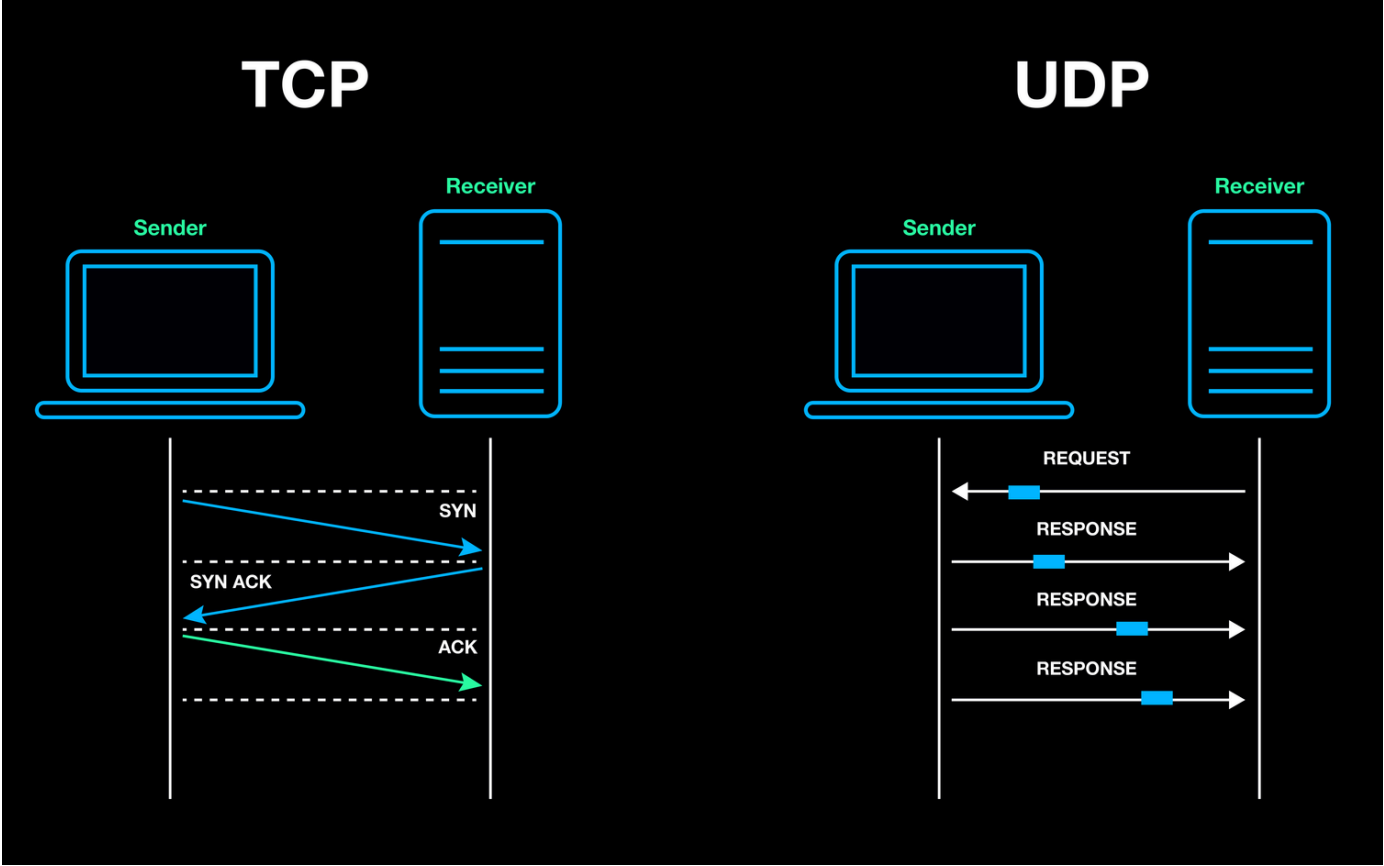
Сетевая модель OSI. Что же дальше?

Теперь, познакомившись с моделью OSI, перейдем к модели TCP/IP. Эта модель почти полностью повторяет эталонную, но имеет свои особенности.



- Первый уровень – канальный. В данной модели он объединяет L1 и L2 уровни модели OSI.
- Второй уровень – межсетевой. Он идентичен сетевому уровню L3 модели OSI.
- Третий уровень – транспортный. Он идентичен транспортному уровню L4 модели OSI.
- Четвертый уровень – прикладной. Он объединяет L5 – L7 уровни модели OSI.

TCP/UDP



TCP/IP

Обычно, когда обсуждают OSI и TCP/IP, пытаются сравнить эти модели и найти соответствия между их уровнями. Хотя картинка ранее и выглядит как сравнение, напрямую мы не будем этого делать, потому что то, как соотносятся их уровни, не особо важно для нас. Да и «многоуровневость» как таковая «может быть вредной», если верить [RFC 3439](#).

Ключевые протоколы модели TCP/IP, очевидно, TCP и IP. Всё остальное в этой модели описано довольно туманно, и даже авторы учебников и справочников по сетям не могут сойтись на чётких названиях и количестве уровней в этой модели.

Есть ли физический уровень под уровнем сетевого доступа? Нужно ли вообще называть уровень сетевого доступа так, или это всё же «канальный уровень»? Вокруг TCP/IP много таких вопросов. Однако, всё это не так важно, потому что основная суть стека очень простая.

Предположим, есть два приложения в сети, которые хотят обменяться данными. Пусть один из них сгенерирует сообщение. Не важно, какое именно. Это может быть HTTP-запрос, а может и нет. Это сообщение генерируется на уровне приложения.

message

TCP/IP

Дальше оно передаётся на транспортный уровень, где к нему добавляется TCP или UDP-заголовок. Этот заголовок содержит информацию о портах, которые используются приложениями, и некоторую другую информацию (но о ней чуть позже). В результате получается то, что обычно называется датаграммой или сегментом.



Затем, сегмент передаётся на межсетевой уровень. Там к нему добавляется IP-заголовок, содержащий адреса компьютеров, на которых запущены приложения, участвующие в этом обмене данными. Результат объединения сегмента и IP-заголовка обычно называется пакетом.

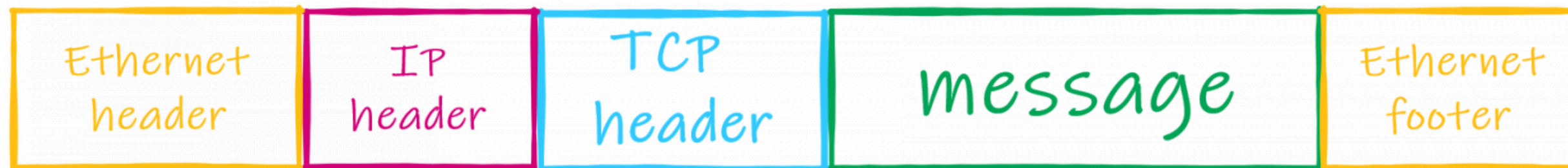
Справедливости ради, нет чёткой разницы между датаграммами, сегментами и пакетами. В большинстве случаев для простоты они все называются «пакетами». Термины «датаграмма» и «сегмент» используются, только когда нужно явно указать тип протокола, с которым мы имеем дело.



TCP/IP

Наконец, пакет передаётся на канальный уровень, где кодируется и уходит по проводам в сторону точки назначения. Протоколы на этом уровне так же добавляют свой заголовок в каждый пакет, и в результате получается кадр (или «фрейм» – пер.).

Кадры обычно содержат не только заголовок, но и окончание. Например, Ethernet-кадр для нашего сообщения может выглядеть примерно так:

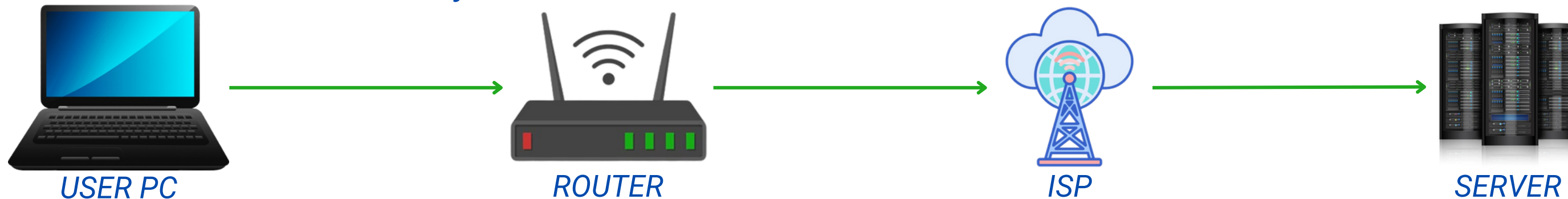


Когда кадр передаётся от одного промежуточного узла сети к другому, эти узлы разбирают кадр на части, проверяют IP-заголовок пакета, определяют что с ним делать, а затем создают новый кадр для этого пакета и передают его следующему узлу.

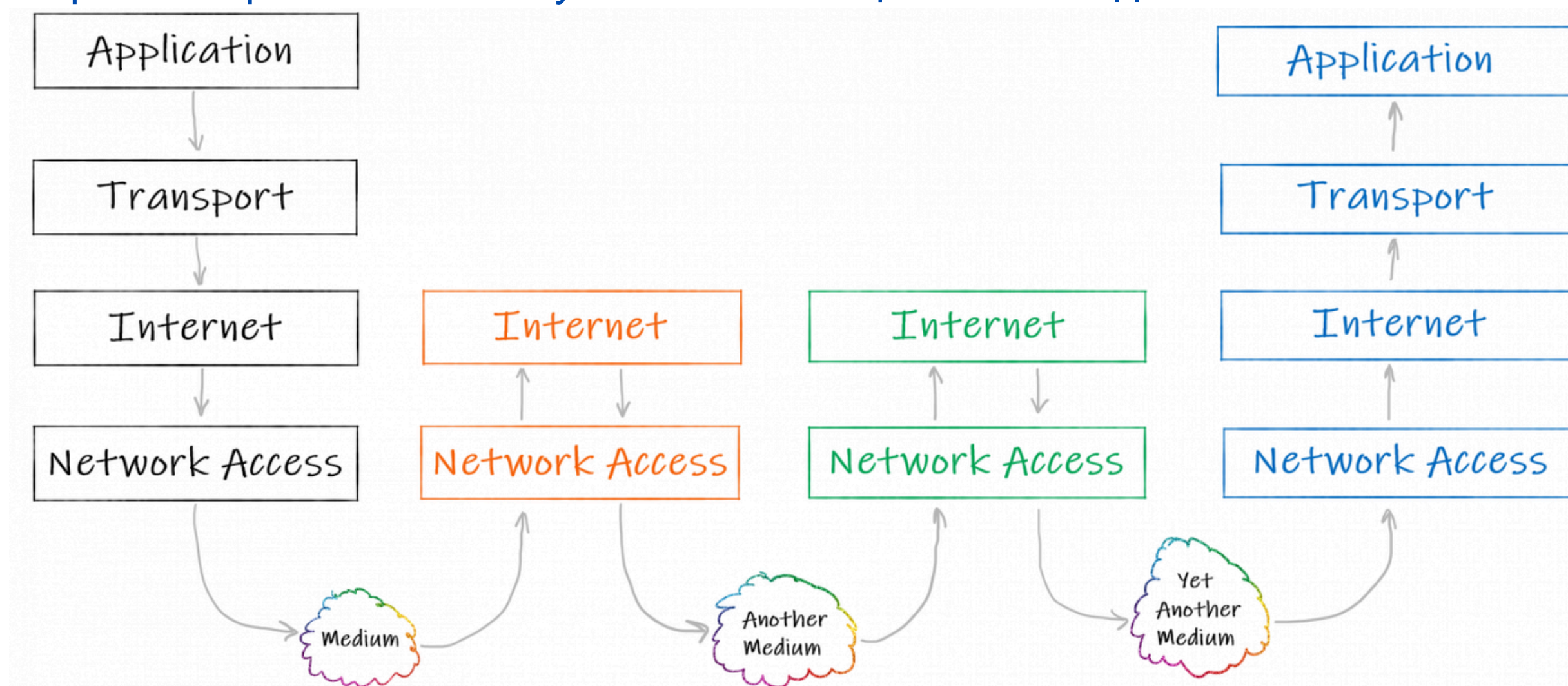
TCP/IP

Когда кадр передаётся от одного промежуточного узла сети к другому, эти узлы разбирают кадр на части, проверяют IP-заголовок пакета, определяют что с ним делать, а затем создают новый кадр для этого пакета и передают его следующему узлу.

Иными словами, когда путь сообщения выглядит как-то так:



То работа протоколов на пути этого сообщения выглядит так:



В начале и конце передачи участвуют протоколы всех уровней TCP/IP-стека. Промежуточные же ноды обычно работают только с протоколами канального и межсетевого уровней

TCP и UDP

Транспортный уровень модели TCP/IP основан на двух китах: Transmission Control Protocol и User Datagram Protocol. Есть и другие протоколы на этом уровне (например, QUICK), но они не так часто используются.

Протоколы транспортного уровня используются для адресации пакетов с порта приложения отправителя на порт приложения получателя. Более того, протоколы этого уровня не знают ничего про различия в узлах сетей. Всё, что им требуется знать про адресацию, это то, что есть приложение, отсылающее сообщение, и оно для отправки использует какой-то порт. И приложение, которое получает сообщение, тоже использует какой-то порт. Основная «адресация внутри Интернета» же реализована на межсетевом уровне, и будет описана далее.

UDP куда «легче» и проще, чем TCP, но в то же время UDP не такой надёжный, как TCP. Чтобы посмотреть на разницу между ними поподробнее, давайте начнём с User Datagram Protocol.

User Datagram Protocol (UDP)

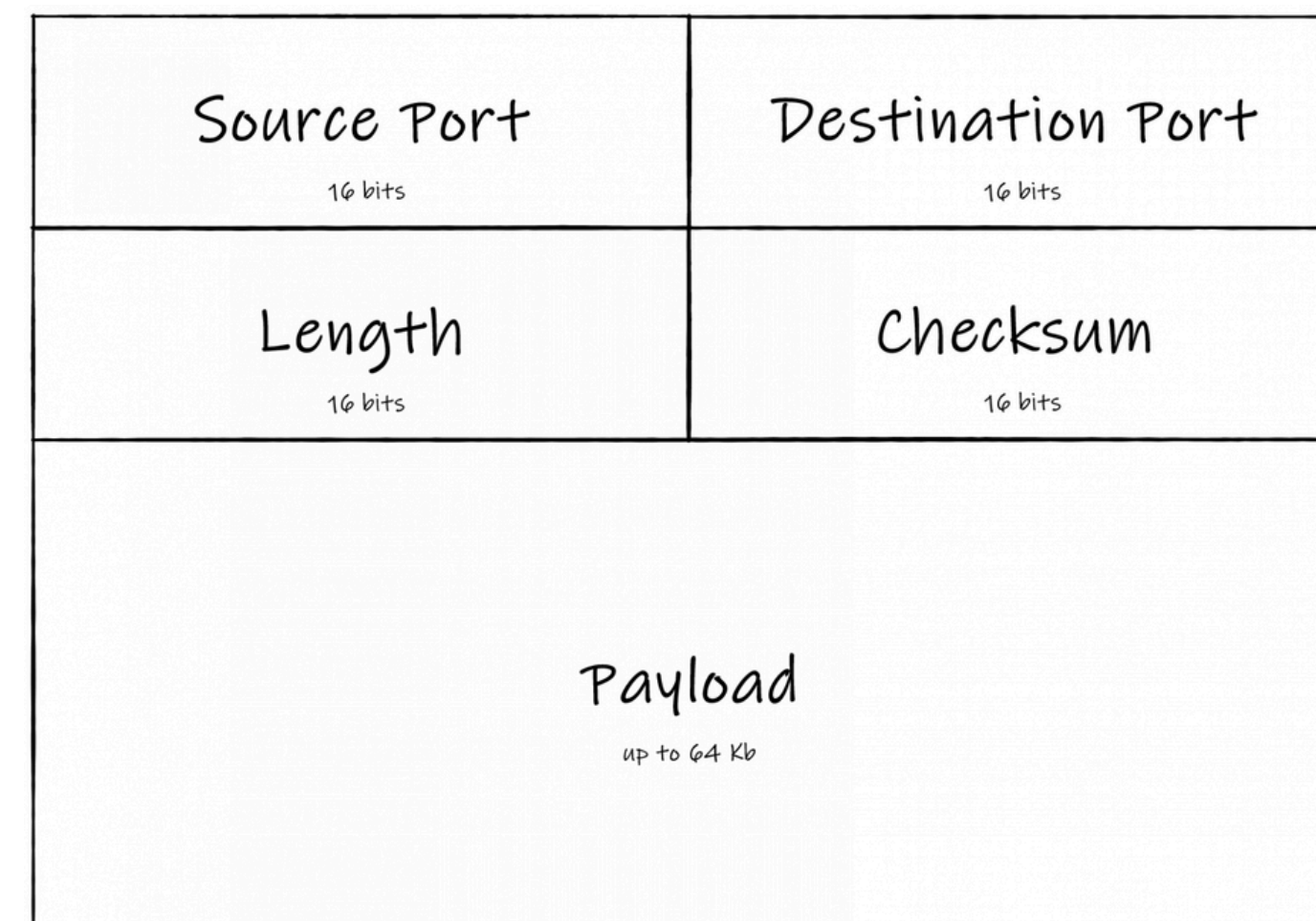
Этот протокол используется для «связи без установки соединения» (connectionless communication – пер.). Один узел сети просто отсылает пакеты, адресуя их другому узлу. Отправитель не знает ничего о том, готов ли получатель к приёму пакетов, и вообще, существует ли этот получатель. Отправитель также не ждёт какого-либо подтверждения о том, что получатель принял предыдущие пакеты.

Пакеты, передаваемые с помощью UDP, иногда называются датаграммами. Этот термин обычно используется тогда, когда важно подчеркнуть, что пакет передаётся без установки соединения.

Заголовок UDP-пакета состоит из 8 байт, которые включают в себя:

- порт отправителя;
- порт получателя;
- длину датаграммы;
- контрольную сумму.

Вот так просто. Типичная UDP-датаграмма выглядит так:



Подытожим

	ТСР	UDP
Состояние соединения	Требуется установленное соединение для передачи данных (после завершения передачи должно быть закрыто)	Протокол без обязательного соединения и требований к открытию, поддержанию или прерыванию соединения
Гарантия доставки	Может гарантировать доставку данных получателю	Нет гарантии доставки данных получателю
Повторная передача данных	Повторная передача пакетов в случае потери одного из них	Нет повторной передачи потерянных пакетов
Проверка ошибок	Выполняется полная проверка ошибок	Действует базовый механизм проверки ошибок. Использует вышестоящие протоколы для проверки целостности
Метод передачи	Данные считываются как поток байтов, информация передается по границам сегментов	У пакетов есть сформированные границы. Пакеты отправляются по отдельности и проверяются на целостность при получении
Сферы применения	Используется для передачи, сообщений электронной почты, HTML - страниц браузеров	Подходит для видеоконференций, потокового вещания, DNS, VoIP, IPTV

Клиент-серверная модель

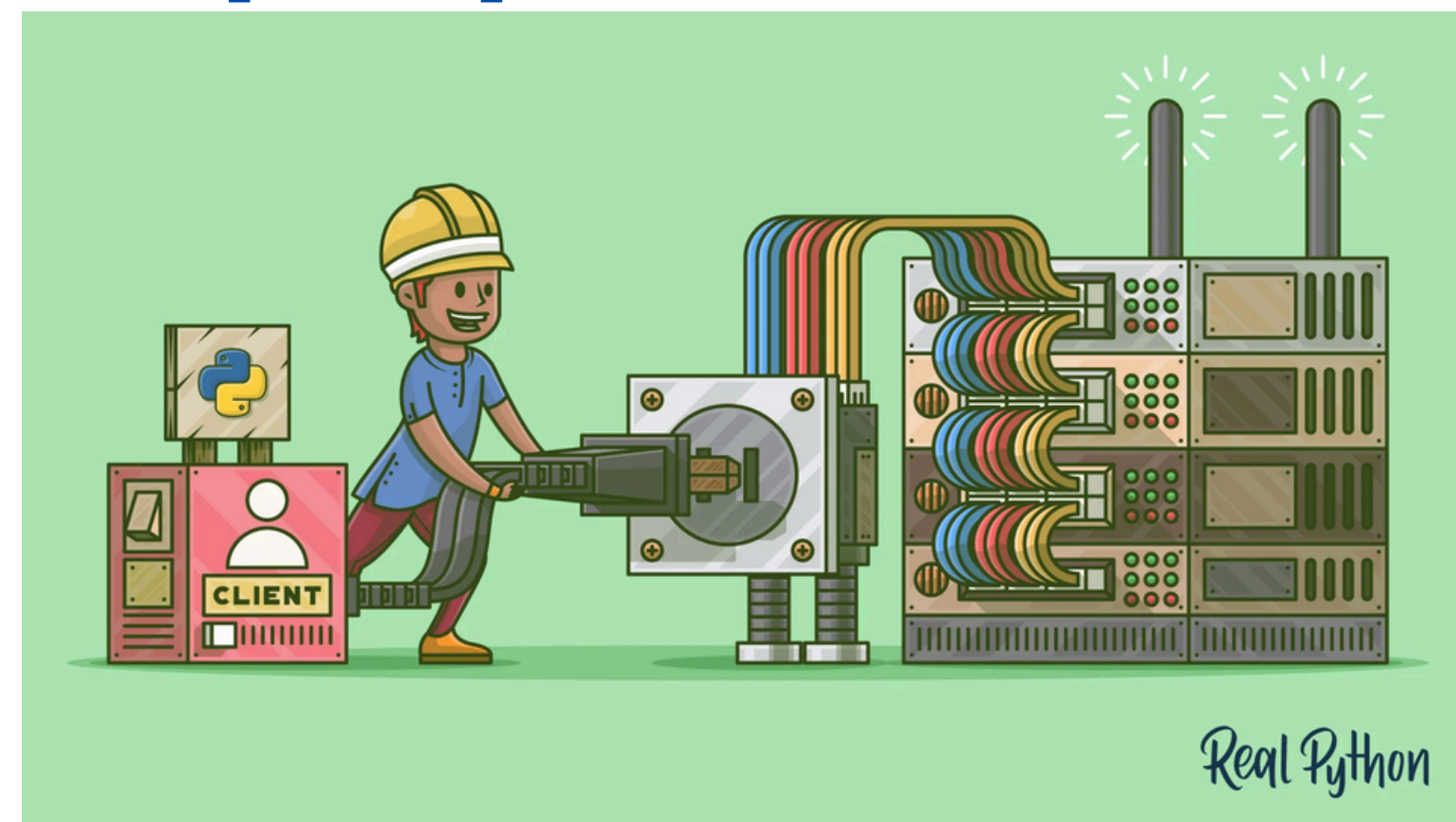


Клиент-серверная модель

Про это мы с вами говорили на самом первом занятии, так что вы легко сможете взять информацию из той презентации, но я нашел еще интересную статью (с картинками!!) и вот ссылка на нее:

<https://habr.com/ru/articles/495698/>

Socket/Эхо-клиент и эхо-сервер



История сокетов

История у сокетов давняя. Их применение началось с ARPANET в 1971 году и продолжилось в 1983-м, когда в операционной системе Berkeley Software Distribution (BSD) появился API под названием «сокеты Беркли».

В 1990-х годах вместе со Всемирной паутиной возникло и сетевое программирование. Преимущества сокетов и первых подключаемых сетей применялись не только в веб-серверах и браузерах — широко стали применяться клиент-серверные приложения разных типов и размеров.

Базовые протоколы API сокетов развивались многие годы, появились и новые, а низкоуровневый API остался прежним.

Самые распространённые сегодня приложения с сокетами — это клиент-серверные приложения, где одна сторона действует как сервер и ожидает подключения клиентов. А конкретнее, сосредоточимся на API сокетов для интернет-сокетов. Иногда их называют сокетами Беркли, или сокетами BSD. Есть и сокеты домена Unix, которые используются для взаимодействия между процессами внутри только одного компьютера.

API сокетов

В модуле `socket` есть интерфейс к API сокетов Беркли.

Вот основные функции и методы этого API:

- `.socket()`
- `.bind()`
- `.listen()`
- `.accept()`
- `.connect()`
- `.connect_ex()`
- `.send()`
- `.recv()`
- `.close()`

В Python имеется удобный и последовательный API, напрямую сопоставленный с системными вызовами, то есть аналогами функций из списка выше на C. Далее вы узнаете, как эти функции используются вместе.

Кроме того, в стандартной библиотеке Python есть классы, которые упрощают применение этих функций.

TCP-сокеты

С помощью **socket.socket()** создается объект сокета с указанием типа сокета **socket.SOCK_STREAM**. При этом по умолчанию применяется протокол управления передачей (TCP).

Почему именно TCP? Как мы уже знаем согласно ранее изученного:

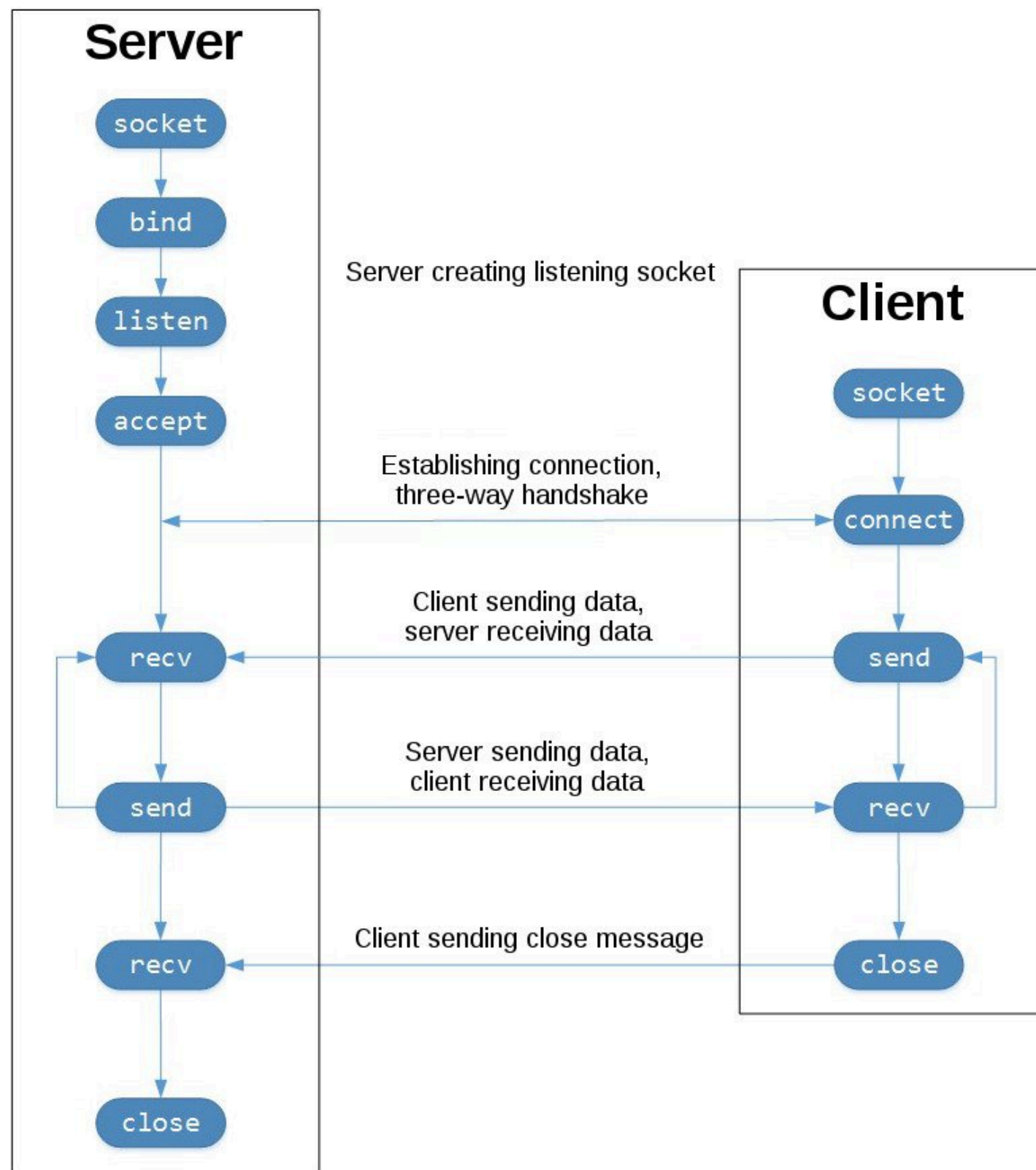
- TCP надёжен. Отброшенные в сети пакеты обнаруживаются и повторно передаются отправителем.
- Данные доставляются с сохранением порядка очерёдности. В приложении данные считываются в порядке их записи отправителем.

Для сравнения: сокеты, которые создаются через **socket.SOCK_DGRAM** протокола пользовательских датаграмм (UDP) ненадёжны: данные могут считываться получателем с изменением порядка очерёдности записей отправителя. Почему это важно? Сети — это система негарантированной доставки. Нет гарантии, что данные дойдут до места назначения или что отправленные данные будут получены.

Сетевые устройства — маршрутизаторы и коммутаторы — также обладают конечной полосой пропускания и собственными, системными ограничениями. Как на клиентах и на серверах, у них есть процессоры, память, шины и интерфейсные буферы пакетов. С TCP при этом не нужно беспокоиться о потере пакетов, поступлении данных с изменением порядка очерёдности пакетов данных, а также о других подводных камнях.

TCP-сокеты

Чтобы разобраться лучше, ознакомимся с последовательностью вызовов API сокетов и с потоком данных TCP. Ниже слева сервер, а справа клиент:



Поток TCP-сокетов. В центре изображения показан обмен данными между клиентом и сервером с помощью вызовов `.send()` и `.recv()`.

Начиная с верхнего левого угла, показаны серверные вызовы API на сервере, которые настраивают «прослушиваемый» сокет:

- `socket()`
- `.bind()`
- `.listen()`
- `.accept()`

Этим сокетом, как следует из его названия, прослушиваются подключения от клиентов. Чтобы принять или завершить такое подключение, на сервере вызывается `.accept()`.

А чтобы установить подключение к серверу и инициировать трёхэтапное рукопожатие, на клиенте вызывается `.connect()`. Процесс рукопожатия важен, ведь он гарантирует доступность каждой стороны подключения в сети, то есть то, что клиент может связаться с сервером, и наоборот. Возможно, только один хост, клиент или сервер может связаться с другим.



Эхо-клиент и эхо-сервер

Теперь, когда мы узнали об API сокетов и взаимодействии клиента и сервера, мы готовы создать свои первые клиент и сервер. Начнём с примера, где полученное сервером сообщение просто возвращается клиенту, как эхо.